Speculative Decoding for Large Language Models: A Case Study

Evaluation of n-gram and EAGLE-3 speculative decoding, plus training an EAGLE-3 drafter from scratch

Bulat Sharipov, DS-01
Innopolis University

Motivation

In this case study I wanted to learn and investigate speculative decoding for large language models. Speculative decoding is an acceleration algorithm inspired by the observation that autoregressive generation is mostly memory-bound, meaning much of the time is spent loading model weights to GPU compute cores. The method uses a draft model to propose tokens, which are then verified by the target model in a single forward pass.

Experimental Setup (Summary)

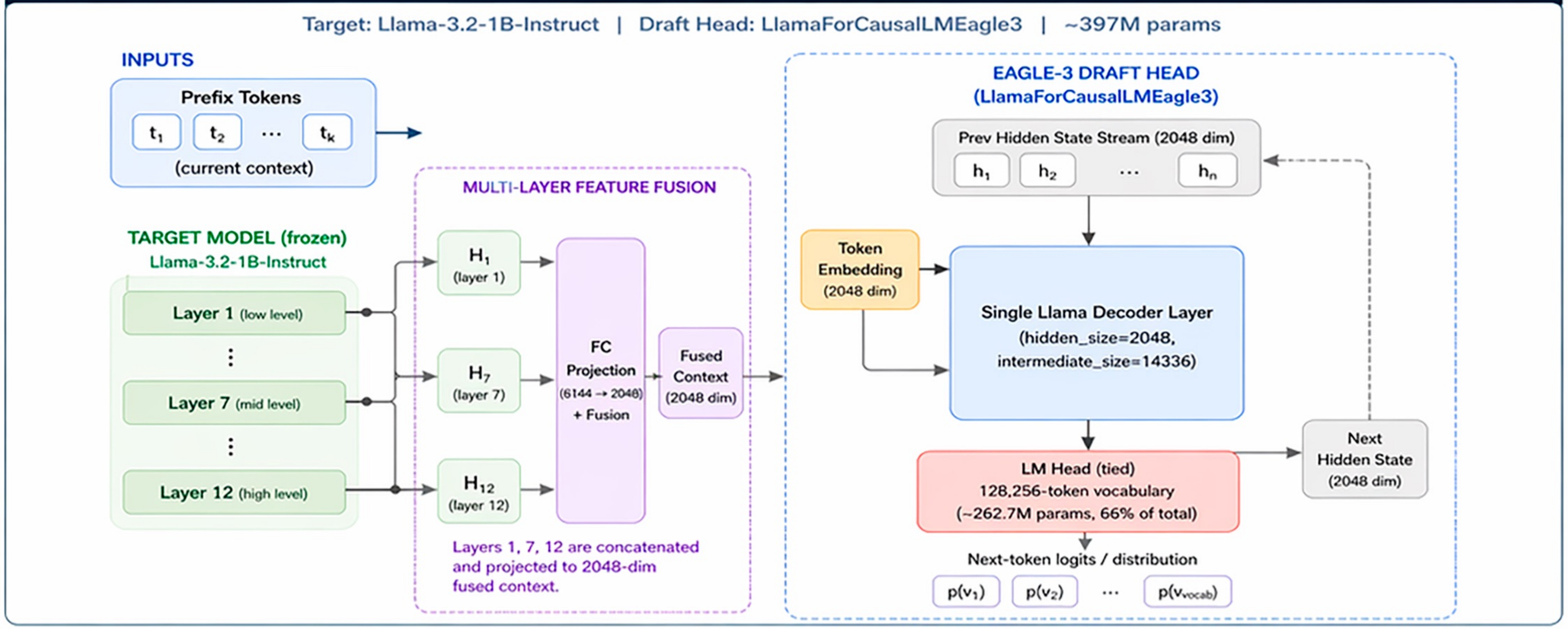
Targets:	Qwen3-8B, A-vibe, Llama-3.2-1B-Instruct
Methods:	baseline, n-gram, EAGLE-3
Datasets:	MT-Bench, ru-arena-hard, UltraChat-200k
Hardware:	RTX 5090

Methodology and Experimental Setup

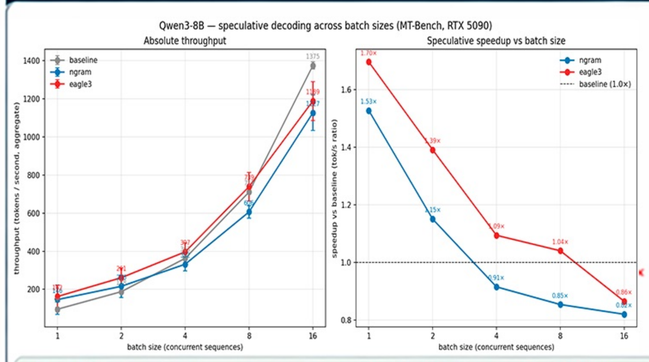
For the first part of the case study I evaluated existing speculative decoding methods across models. I selected Qwen3-8B and A-vibe for evaluation. For generation I compared three methods: naive autoregressive decoding (baseline), an n-gram drafter, and an EAGLE-3 drafter. Publicly available EAGLE-3 draft models were downloaded for both targets. The dataset for Qwen3-8B was MT-Bench, a benchmark for evaluating multi-turn conversations in LLMs. For A-vibe I used ru-arena-hard, a Russian translation of Arena-Hard.

In the second part of the study I trained my own drafter model from scratch. Following the NVIDIA guide, I used Llama-3.2-1B-Instruct as the target model and trained an EAGLE-3 drafter for it using the provided scripts. The resulting drafter had about 400M parameters and was largely bottlenecked by the language modeling head.

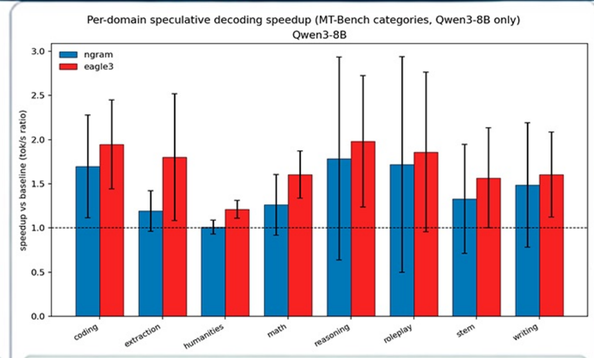
EAGLE-3 Drafter Architecture



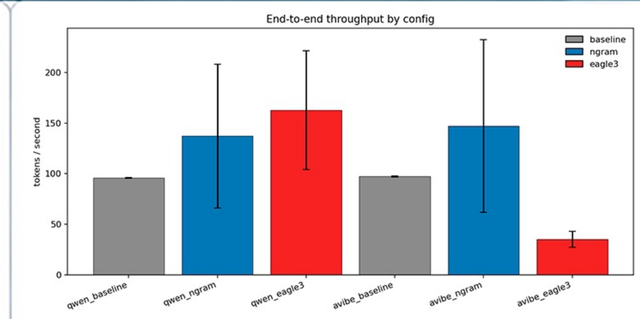
Results



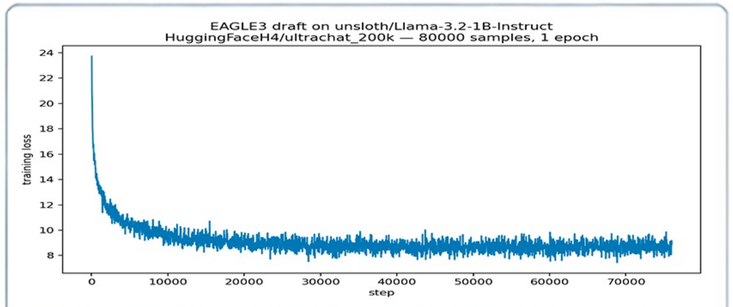
EAGLE-3 gives the strongest speedup at low batch sizes on Qwen3-8B, but gains diminish as batch size increases.



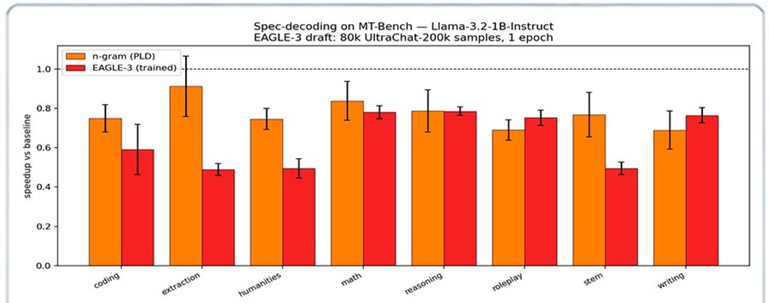
On Qwen3-8B, EAGLE-3 consistently outperforms n-gram across most MT-Bench domains.



Qwen3-8B benefits strongly from speculative decoding, while the A-vibe EAGLE drafter underperforms, suggesting poor acceptance with the target model.



The trained EAGLE-3 drafter for Llama-3.2-1B shows stable optimization during training.



For the 1B target, speculative decoding does not help on RTX 5090; both n-gram and EAGLE-3 are slower than baseline.

Conclusion

Speculative decoding can provide significant speedups for large language models. Both n-gram and EAGLE-3 improve throughput when the target model is sufficiently large and memory-bound. However, speculative decoding must be applied carefully.

First, it is generally not useful for small models. Small targets already run quickly and are less memory-bound, so the overhead of drafting and verification can outweigh any benefit. It is also difficult to train good drafter models in the very small-model regime.

Second, as batch size increases, the gains from speculative decoding largely vanish and can turn into overhead. In that regime generation becomes more compute-bound, so speculative decoding no longer addresses the primary bottleneck.

Finally, drafter quality matters. As seen in the A-vibe experiment, a poorly aligned drafter may fail to achieve good acceptance rates and can hurt end-to-end throughput.